

▽ AI활용프로그래밍 Week 12 — 필터링·정렬 (따라하기 노트북)

- 이 노트북은 **PPT 예시 코드를 그대로 따라 실행**할 수 있게 정리한 버전입니다.
- 각 Step은 **독립적으로 실행**되도록 구성했습니다(앞 셀을 안 돌려도 실행 가능).
- `Run All` 보다는 **Step 1 → Step 10 순서대로** 실행하는 것을 권장합니다.

학습 목표

- Pandas DataFrame에서 **조건 필터링(Boolean indexing)** 을 수행한다.
- **정렬(sort_values / sort_index)** 과 **Top-N(nlargest/nsmallest)** 를 사용할 수 있다.
- `loc/iloc`, `query()`, 결측치 처리, `SettingWithCopyWarning` 회피를 익힌다.

▽ Step 0. 실행 환경 준비

아래 셀을 실행해서 환경(버전)이 잘 준비되었는지 확인하세요.

```
import sys
import pandas as pd

print('python', sys.version.split()[0])
print('pandas', pd.__version__)
```

▽ 워밍업: 원하는 행만 골라내기

해야 할 것

- `score`가 **80 이상**인 행만 선택해 보세요.

아래 셀은 **TODO(실습용)** 입니다.

```
import pandas as pd

df = pd.DataFrame({'name':['a','b','c'], 'score':[70,85,90]})
df

# TODO: score가 80 이상인 행만 선택
# 힌트: df[조건]
```

▽ 정답 예시

```
import pandas as pd

df = pd.DataFrame({'name':['a','b','c'], 'score':[70,85,90]})
df[df['score'] >= 80]
```

▽ Step 1. Boolean indexing 기본 (df[조건])

해야 할 것

- 조건을 먼저 변수(`cond`)로 만든다.
- `cond` (True/False)를 출력해 확인한다.
- `df[cond]`로 필터한다.

체크

- `b`, `c` 행만 남는다(`score >= 80`).

```
import pandas as pd

df = pd.DataFrame({'name':['a','b','c'], 'score':[70,85,90]})
```

```
cond = df['score'] >= 80
print(cond)

high = df[cond]
print(high)
```

✓ Step 2. 복합 조건(&, |) — 괄호가 중요

해야 할 것

- Pandas에서 복합 조건은 `and/or`가 아니라 `& / |` 를 쓴다.
- 각 조건은 **괄호로 감싸기**: `df[(cond1) & (cond2)]`

체크

- `80 <= score < 90` 인 행만 선택된다.

```
import pandas as pd

df = pd.DataFrame({
    'name': ['a', 'b', 'c', 'd'],
    'region': ['A', 'B', 'A', 'C'],
    'score': [70, 85, 90, 88]
})

cond1 = df['score'] >= 80
cond2 = df['score'] < 90
print(df[(cond1) & (cond2)])

# OR(|) 예시: region이 A 또는 B
print(df[(df['region'] == 'A') | (df['region'] == 'B')])
```

✓ Step 3. 자주 쓰는 조건: `isin` / `between` / `str.contains`

해야 할 것

- `isin`: 특정 값 집합
- `between`: 범위
- `contains`: 문자열 포함 (`na=False`로 결측치 안전)

```
import pandas as pd

df = pd.DataFrame({
    'region': ['A', 'B', 'C', None],
    'score': [70, 85, 90, 88],
    'name': ['kim', 'lee', 'park', 'choi']
})

print(df[df['region'].isin(['A', 'B'])])
print(df[df['score'].between(80, 90)])
print(df[df['name'].str.contains('k', na=False)])
```

✓ Step 4. 정렬 `sort_values` (값 기준)

해야 할 것

- `ascending=False`면 내림차순
- 정렬 후 `head()`로 Top-N 보기

```
import pandas as pd

df = pd.DataFrame({'name': ['a', 'b', 'c'], 'score': [70, 85, 90]})

print(df.sort_values('score', ascending=False))
print(df.sort_values('score', ascending=False).head(2))
```

▼ Step 5. Top-N: `nlargest` / `smallest`

해야 할 것

- 전체 정렬 없이 상/하위 뽑기
- 동점(ties) 처리도 확인

```
import pandas as pd

df = pd.DataFrame({'name': ['a', 'b', 'c', 'd'], 'score': [70, 85, 90, 85]})

print(df.nlargest(2, 'score'))
print(df.smallest(2, 'score'))
```

▼ Step 6. 필터 + 정렬 파이프라인(한 줄)

해야 할 것

- `df[조건]` 뒤에 `.sort_values()`
- 중간 확인은 `shape`, `head()`로

```
import pandas as pd

df = pd.DataFrame({
    'name': ['a', 'b', 'c', 'd'],
    'score': [70, 85, 90, 88]
})

result = (df[df['score'] >= 80]
          .sort_values('score', ascending=False))

print(result.head(3))
```

▼ Step 7. 결측치(NaN) 기본

해야 할 것

- `isna()`로 결측 확인
- `dropna()` / `fillna()` 사용

```
import pandas as pd

df = pd.DataFrame({'score': [70, None, 90]})

print(df['score'].isna())
print(df.dropna())
print(df.fillna(0))
```

▼ Step 8. `loc` / `iloc`로 행·열 선택

해야 할 것

- `loc`: 라벨 기반(조건 + 컬럼 선택에 자주 사용)
- `iloc`: 위치 기반

```
import pandas as pd

df = pd.DataFrame({'name': ['a', 'b', 'c'], 'score': [70, 85, 90]})

print(df.loc[df['score'] >= 80, ['name', 'score']])
print(df.iloc[0, 1])
```

▼ Step 9. `query()`로 조건을 문자열로

해야 할 것

- 조건이 길면 가독성 개선
- 팀에서는 읽기 쉬운 방식을 선택

```
import pandas as pd

df = pd.DataFrame({'name': ['a', 'b', 'c'], 'score': [70, 85, 90]})

print(df.query('score >= 80 and score < 90'))
```

참고: 문자열 필터링 (str) 메서드

```
df[df['name'].str.startswith('k')]
df[df['name'].str.contains('im', na=False)]
```

- (na=False)로 결측치 안전 처리
- 전처리(소문자 통일) 후 필터하면 정확도↑

✓ Step 10. (SettingWithCopyWarning) 피하기

해야 할 것

- 필터 결과는 view/복사본이 모호할 수 있음
- (.copy())로 명확히 복사
- 수정은 (loc)로 하는 습관

```
import pandas as pd

df = pd.DataFrame({'name': ['a', 'b', 'c'], 'score': [70, 85, 90]})

sub = df.loc[df['score'] >= 80].copy()
sub['level'] = 'high'
print(sub)
```

성능/가독성 팁

- 조건은 먼저 작게 만들어((cond)로) 확인
- 필요한 컬럼만 선택해서 처리
- 정렬은 최소 횟수로
- 중간 결과를 (head()) / (shape)로 자주 확인

✓ 미니 예제: 랭킹 만들기(스켈레톤)

요구: 조건(예: (region == 'A'))으로 필터 후 (score) 기준 내림차순 정렬, Top 10 출력

```
import pandas as pd

df = pd.DataFrame({
    'name': ['kim', 'lee', 'park', 'choi', 'jung', 'han', 'song', 'yoon', 'lim', 'oh', 'na'],
    'region': ['A', 'A', 'B', 'A', 'C', 'A', 'B', 'A', 'C', 'A', 'B'],
    'score': [91, 88, 77, 95, 66, 84, 79, 90, 72, 93, 85]
})
df

# TODO: region이 'A'인 행만 필터 → score 내림차순 정렬 → Top 10
```

✓ 정답 예시

```
import pandas as pd

df = pd.DataFrame({
    'name': ['kim', 'lee', 'park', 'choi', 'jung', 'han', 'song', 'yoon', 'lim', 'oh', 'na'],
```

```

        'region':['A','A','B','A','C','A','B','A','C','A','B'],
        'score':[91, 88, 77, 95, 66, 84, 79, 90, 72, 93, 85]
    })

result = (df[df['region'] == 'A']
          .sort_values('score', ascending=False)
          .head(10))

result

```

With AI: EDA 프롬프트 템플릿(복사해서 사용)

- 역할: 너는 데이터 분석가다.
- 데이터: `df.head()`, `df.info()`, `df.describe()` 결과 제공
- 목표: (예: 상위 고객 찾기, 성적 상위 10명 뽑기)
- 요청:
 1. 필요한 필터 조건 제안
 2. Pandas 코드
 3. 결과 해석
 4. 데이터 품질 체크(결측/이상치)

✓ 실습 1: 조건 필터 + Top-N

문제: 데이터에서 조건을 2개 이상 사용해 필터링한 뒤 Top 5를 출력하라. 예: `(score>=80) AND (region in ['A','B'])`

```

import pandas as pd

df = pd.DataFrame({
    'name':['a','b','c','d','e','f','g'],
    'region':['A','B','A','C','B','A','C'],
    'score':[70,85,90,88,82,95,60]
})
df

# TODO: (score>=80) AND (region in ['A','B']) 필터 후 score 내림차순 Top 5

```

✓ 정답 예시

```

import pandas as pd

df = pd.DataFrame({
    'name':['a','b','c','d','e','f','g'],
    'region':['A','B','A','C','B','A','C'],
    'score':[70,85,90,88,82,95,60]
})

cond = (df['score'] >= 80) & (df['region'].isin(['A','B']))
result = df[cond].sort_values('score', ascending=False).head(5)
result

```

✓ 실습 2: 디버깅 — 괄호/연산자 실수

Pandas에서는 `and/or` 대신 `&/|` 를 사용해야 합니다.

```

import pandas as pd

df = pd.DataFrame({'name':['a','b','c'], 'score':[70,85,90]})

try:
    # 잘못된 예(에러 또는 이상한 결과)
    bad = df[df['score'] >= 80 and df['score'] < 90]
    print(bad)
except Exception as e:
    print('에러 발생:', type(e).__name__, e)

# 올바른 예
good = df[(df['score'] >= 80) & (df['score'] < 90)]

```

```
print(good)
```

✓ 실습 3: 리팩터링 — 필터 함수 만들기

분석 코드를 함수로 만들면 재사용성이 좋아집니다.

```
import pandas as pd

def filter_top(df, cond, sort_col, n=5):
    filtered = df[cond] # 조건으로 필터링
    sorted_df = filtered.sort_values(by=sort_col, ascending=False) # 정렬
    return sorted_df.head(n) # 상위 n개 반환

df = pd.DataFrame({
    'name': ['a', 'b', 'c', 'd'],
    'region': ['A', 'B', 'A', 'B'],
    'score': [70, 85, 90, 88]
})

cond = (df['region'] == 'A') & (df['score'] >= 80)
print(filter_top(df, cond, sort_col='score', n=3))
```

연습문제(추가)

1. `score`가 평균 이상인 행만 선택하라.
2. 특정 카테고리만 선택(`isin`)하고, `score`로 내림차순 정렬하라.
3. 문자열 컬럼에서 특정 키워드가 포함된 행을 찾고(`contains`) 개수를 출력하라.
4. Top 3와 Bottom 3를 각각 구하라(`nlargest`/`nsmallest`).

미니 퀴즈(개념 확인)

1. Pandas에서 복합 조건 결합에 쓰는 연산자는?
A. and/or B. &/| C. +/*
2. `df.sort_values('col', ascending=False)`는?
A. 오름차순 B. 내림차순 C. 랜덤
3. 상위 5개를 뽑는 대표 함수는?
A. head(5) B. tail(5) C. sum(5)
4. `df['name'].str.contains('a')`의 용도는?
A. 숫자 합 B. 문자열 포함 필터 C. 정렬
5. `SettingWithCopyWarning`의 의미는?
A. 메모리 부족 B. 뷰/복사본 수정 모호 C. 파일 없음

정답: 1-B, 2-B, 3-A, 4-B, 5-B